

This document is written based on the video series of Neural Network by StatQuest on Youtube. Link is provided as below:

<https://www.youtube.com/playlist?list=PLblh5JKOoLUixGDQs4LFFD--41Vzf-ME1>

NLP Course | For You by Lena Voita

https://lena-voita.github.io/nlp_course.html

Will only start from LSTM onwards. It is recommended to watch since this document is summarized and based on my personal understanding.

General Note / Glossary

1. Parameter = weights and biases
-

RNN and CNN

— Will not be included

Long Short Term Memory (LSTM) - 1997

Original paper:

<https://www.bioinf.jku.at/publications/older/2604.pdf>

Youtube video by Statquest:

<https://www.youtube.com/watch?v=YCzL96nL7j0&list=PLblh5JKOoLUixGDQs4LFFD--41Vzf-ME1&index=16>

Assumption:

- Familiar with **Recurrent Neural Networks** and **Vanishing/Exploding Gradient Problem**

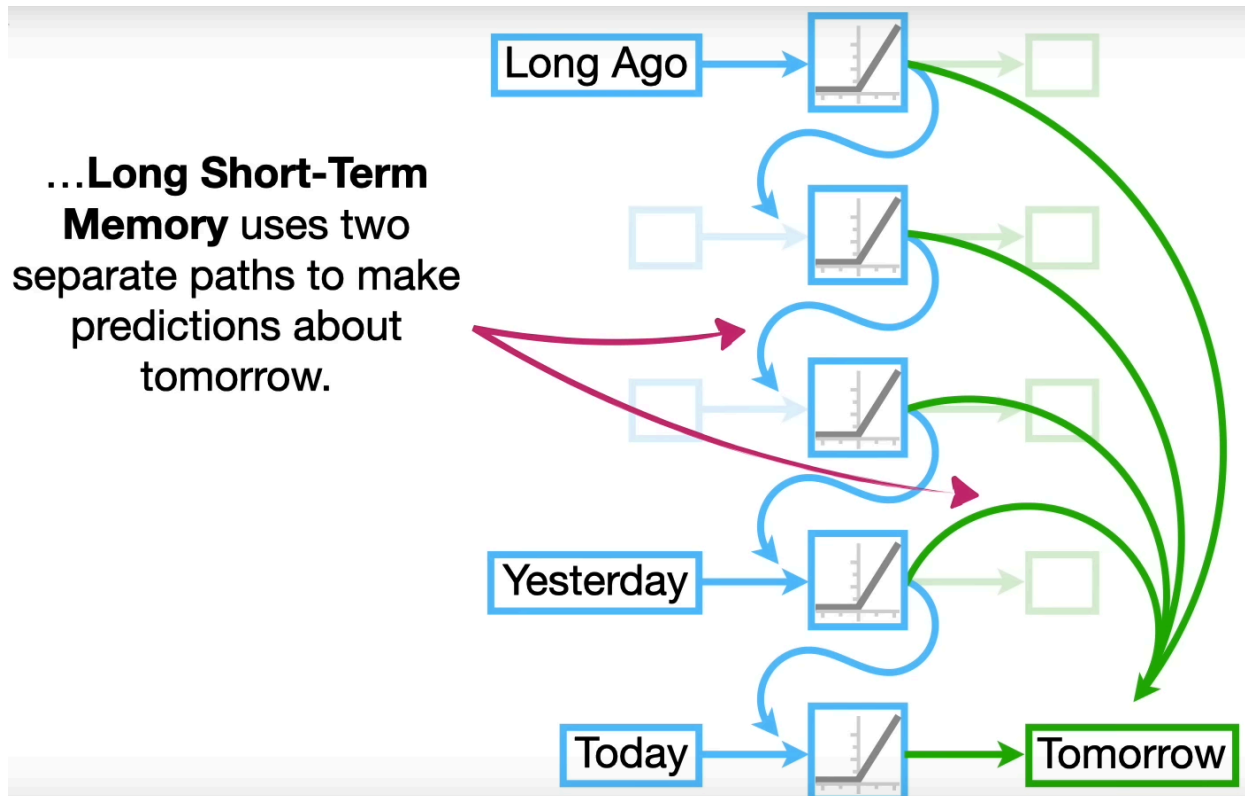
Initial Note:

- This LSTM is built to address the vanishing and gradient problem of basic recurrent neural network
- Consider that RNN uses a single feedback loop/path; only one path from first to final node, the requirement of weights needed to save each and every node is massive.
Example weight of 2: weights required is

$$= \text{Input}_i \times 2^{\text{Num,Unrol}} = \text{Massive (exploding gradient)}$$

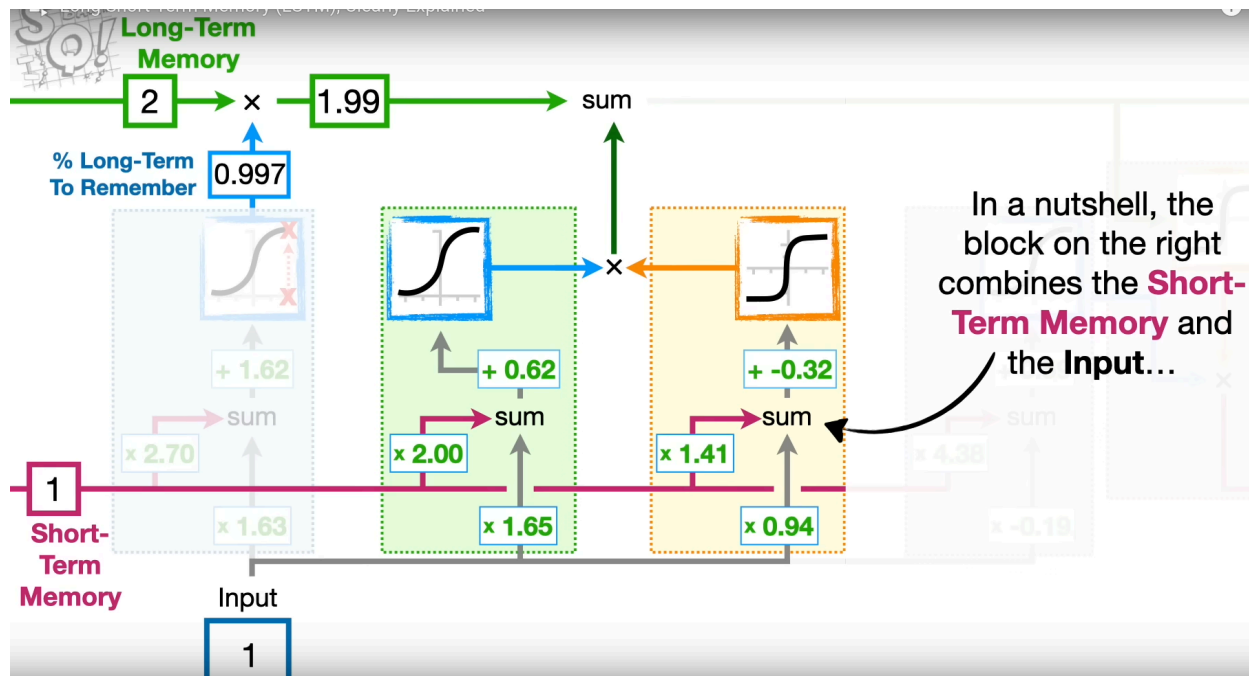
Note:

- LSTM gives the current node information from both one node previous to this, and also from the first initial nodes. This is done by instead of passing the multiplicative of the weights, the output of each weight is directly fed to the output of the layer.



3:32

The path containing old information will be the “long-term memory” and the newest information to the node will be short-term memory.



9:32

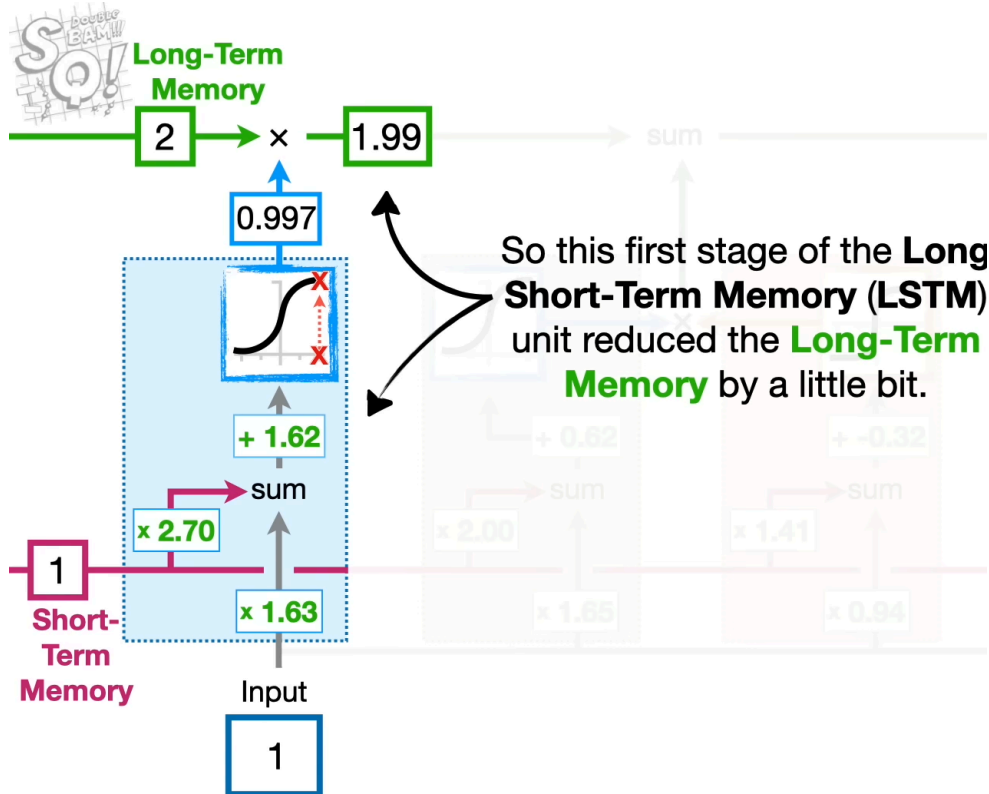
- Top green line: **Cell state**: represents Long-Term Memory
 - The lack of weights in Cell state allows the long-term memory to pass information without saving too many data, avoiding the exploding / vanishing gradient issue.
- Bottom pink line: represents Short-Term Memories
 - Has weights that can directly modify the input
- First stage (Blue block) is the **Forget Gate**
- Second stage (green & yellow) is the **Input Gate** (Long-Term Memory Update)
- Final stage (pink & purple) is the **Output Gate** (Short-Term Memory Update)

Flow:

First stage:

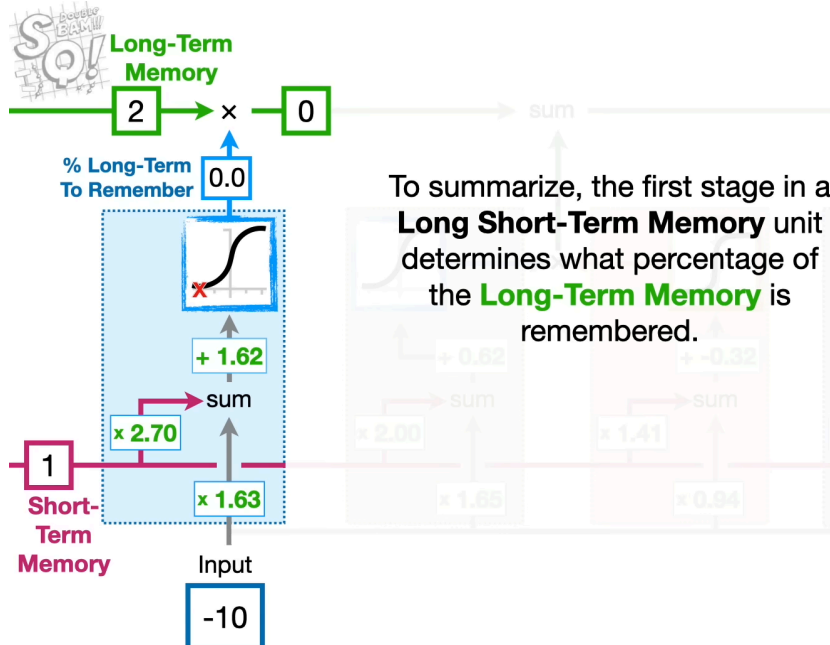
Determines what percentage of the Long-Term Memory is remembered

- If input is more than 1 with short-term of 1 and long-term of 2, the result of the cell is that it slightly reduces the long-term memory to 1.99



8:08

- If input is -10, this particular LSTM cell will compute 0 (means this word will be forgotten)
- Note that in NLP, input -10 has no meaning. It's not necessarily a bad number/word.



8:50

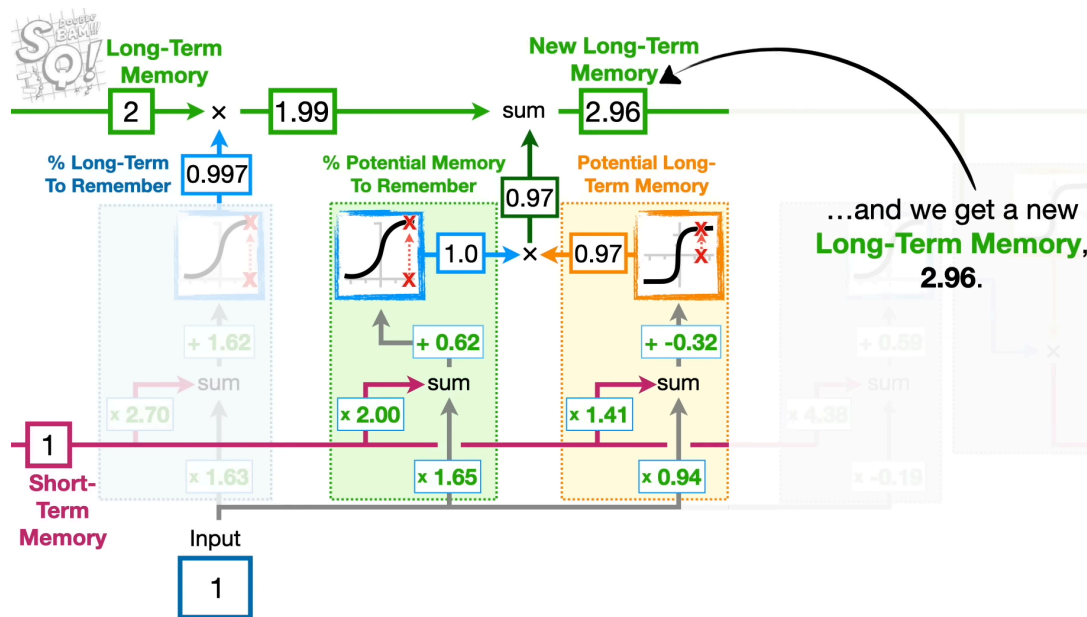
Second stage:

Second stage is used to calculate if and how much of the input and Short-term memory will be stored to the Long-Term memory.

Yellow block on the right – Create a **Potential Long-Term Memory** via combining **Short-Term Memory and Input**

- It uses tanh activation function as it collapses the number to output either 0 or 1

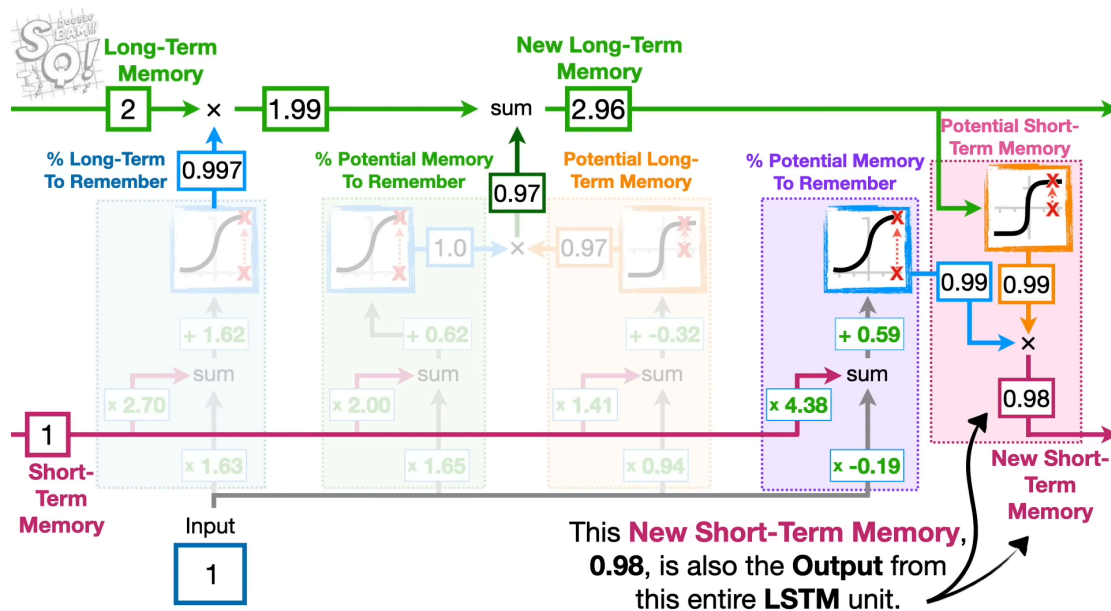
Green block on the left – Determines what **percentage of that Potential Memory** to add to the Long-Term Memory



12:18

Third stage (Final):

Final stage updates the short-term memory



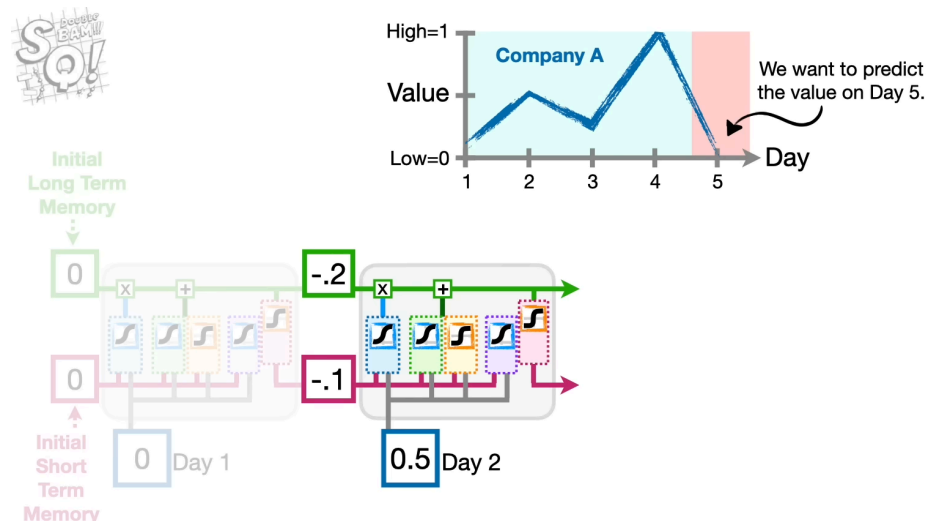
13:56

- Works similarly with Long-Term memory updating

Example:

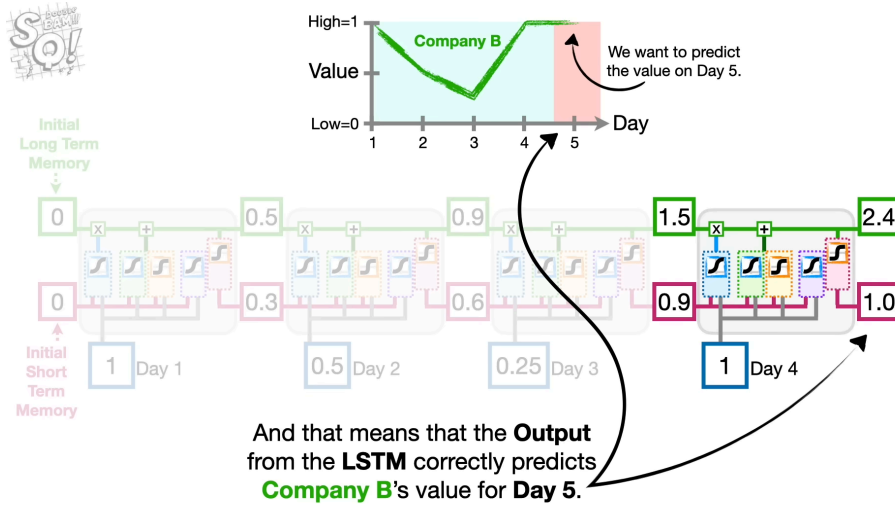
Watch 14:33 for LSTM in action with real data

- 16:44 "Unrolling" (extending) LSTM in one sequence has the same weights and biases
- This allows the model to handle data sequences of different lengths.
- Example of data with 5 days, Each day's data will be inputted as one LSTM's input

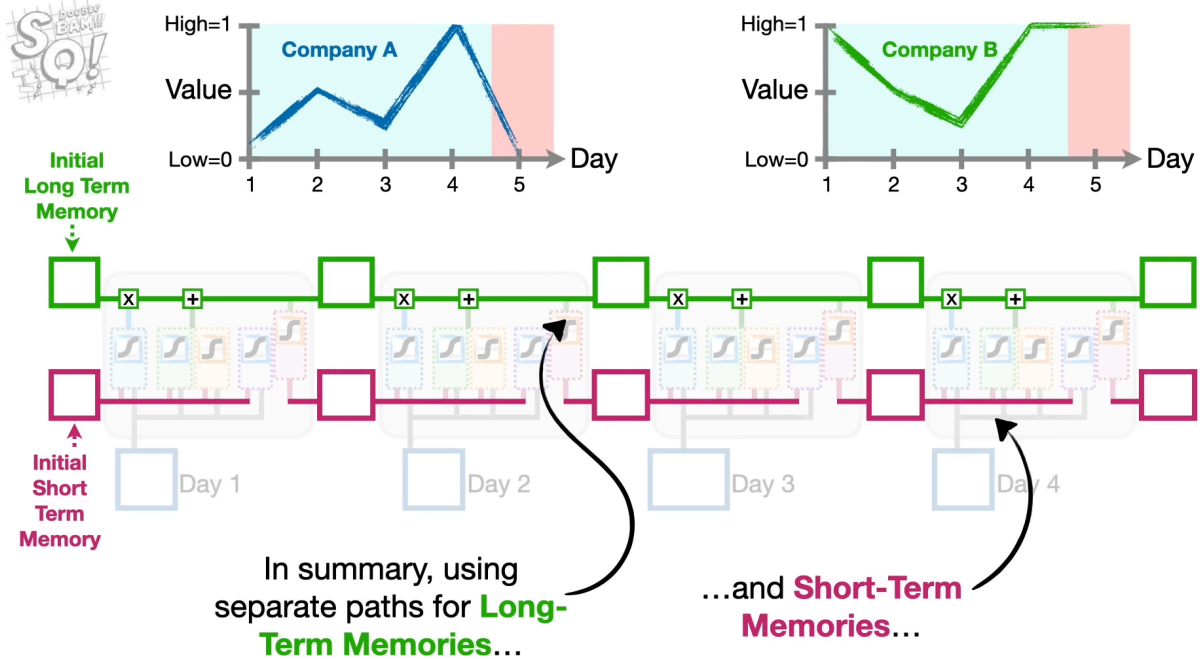


Then the **LSTM** does it's math,
using the exact same **Weights**
and **Biases** as before...

16:52



19:24



19:37

Word Embedding

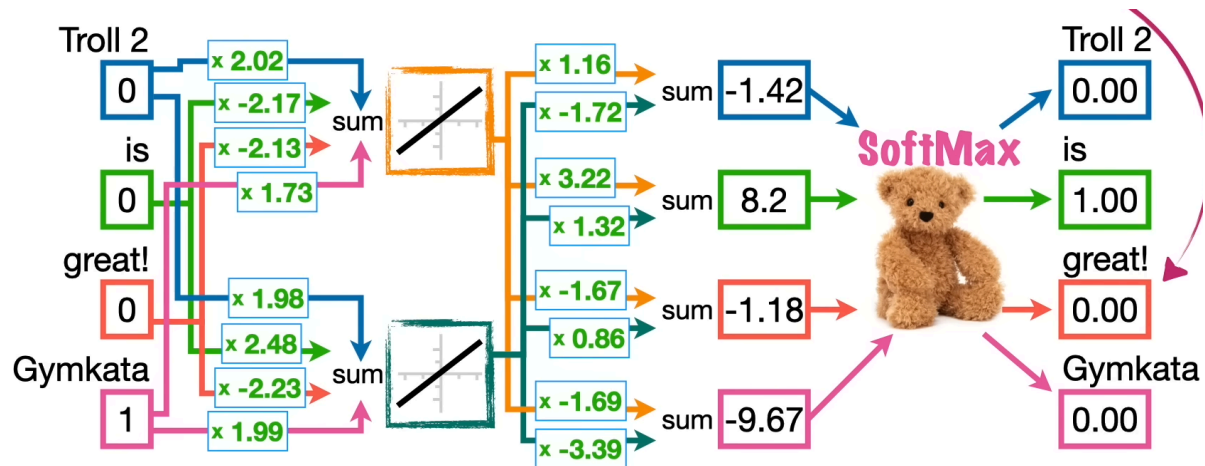
Youtube video by StatQuest:

<https://www.youtube.com/watch?v=viZrOnJclY0&list=PLblh5JKOoLUlxGDQs4LFFD--41Vzf-ME1&index=17>

There are different ways to turn words to vector (words being represented in numbers which can be calculated by computers easily).

- Word2Vec is the umbrella term for methods to optimized in vectorizing words.

The following shows a very basic RNN that predicts the next word where “Gymkata” is input and the model predicted “is” is the next word.



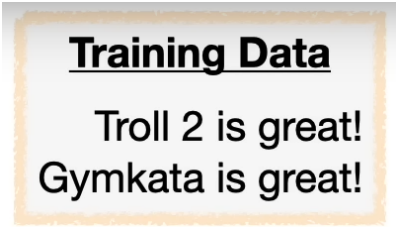
10:35

However, it doesn't learn any context and understand the words.

Word embedding methods

The most basic idea of this is giving the words “representation” via linking them with a number. Think of it like a spiderweb where the nodes connecting the lines are the words, and the lines itself are the weights and biases.

- Know that in practice, people use 100++ activation functions to create a lot of **Embeddings per word**.
- They also use an entire catalogue of data to train them.
- Showing which words occur near which the most.
- As there will be lots of weights and biases (into the billions and beyond), **Negative Sampling** became a practice.
- Below shows some example of word embedding methods. The below is generated with help of ChatGPT4o. The words embedded are arbitrary and irrelevant. Only the embedded result is shown for understanding.



Method	Year		Description	Pros	Cons	Issues Addressed	Example (in Words)
Bag of Words	1950s		BoW represents text as a collection of individual words without considering the order or context of the words. Each document is converted into a vector based on the frequency of words. ~ Each word is represented by a large sparse vector with a single 1 in the position corresponding to the word's index in the vocabulary.	Simple and easy to implement. Effective for basic text classification tasks.	Ignores word order and context. Results in large and sparse vectors for large vocabularies. Cannot capture semantic meaning.	N/A. It is one of the earliest method.	Binary representation of words. "Troll 2" = [0,0,0,1] "is" = [0,0,1,0] "great!" = [0,1,0,0] "Gymkata" = [1,0,0,0] Sentence 1: "Troll 2 is great!" → [1, 0, 1, 0, 1, 1, 1, 0] Sentence 2: "Gymkata is great!" → [0, 1, 1, 0, 1, 1, 1, 0] Numbers are arbitrary.
		Word	Example Embedding	Description			
		company	[1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]	Binary representation indicating the presence of words in the vocabulary.			
		list	[0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]	Each word is represented by a unique index in a large sparse vector			
		numbers	[0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0]	Simple, lacks context and semantic meaning.			
Continuous BoW	2013	surrounding	CBOW is a neural network-based model that predicts the current word given a window of surrounding context words. It captures the semantic meaning of words by learning word embeddings. ~Each word is represented by a dense vector. The embedding captures the context of surrounding words .	Efficient to train. Good at predicting frequent words. Captures semantic relationships between words.	Less effective for infrequent words. Might not capture the specific order of words well.	Addresses the sparsity and lack of semantic meaning in BoW by learning dense word embeddings that capture context.	Given "Troll2" and "great!", the model can predict the middle word is "is" Troll 2 <u>is</u> great!
		Word	Example Embedding	Description			

		company	[-0.45, 0.1, 0.9, 0.7, -0.2, -0.1, ...]	Dense vector representation capturing context from surrounding words.			
		list	[0.3, -0.4, 0.2, 0.5, 0.1, -0.2, ...]	Learned from predicting the current word based on its context.			
		numbers	[-0.1, 0.6, -0.3, -0.4, 0.5, -0.1, ...]	Efficient and effective for frequent words, may not capture rare words well			
Skip-gram	2013	surrounding	Skip-gram is another neural network-based model that predicts surrounding context words given the current word. It is effective for learning word embeddings, especially for capturing rare word semantics. ~ Each word is also represented by a dense vector, but the model is trained to predict the surrounding words given the current word. This method captures rare word semantics well.	Better at capturing meanings of rare words compared to CBOW. Produces high-quality word embeddings that capture semantic and syntactic relationships.	Slower to train compared to CBOW. Requires more training data and computational resources.	Skip-gram improves on CBOW by better capturing the semantics of infrequent words and providing higher-quality word embeddings. ~ Improves on CBOW for rare words	Given "is", the model can predict "Troll 2", "great" and "Gymkata" <u>Troll 2 is great!</u>
		Word	Example Embedding	Description			
		company	[0.2, -0.5, 0.3, -0.1, 0.4, -0.3, ...]	Dense vector representation capturing context words given the current word.			
		list	[-0.3, 0.4, -0.2, 0.3, 0.5, -0.1, ...]	Better at capturing rare word semantics compared to CBOW.			
		numbers	[0.5, -0.4, 0.6, -0.3, 0.2, 0.4, ...]	Produces high-quality embeddings.			
Word2Vec (general method encompassing CBOW and Skip-gram)	2013		Word2Vec is a family of models (including CBOW and Skip-gram) designed to produce word embeddings by training a shallow neural network to predict words based on context.	Efficient and scalable. Produces high-quality word embeddings. Captures both semantic and syntactic relationships.	Requires substantial training data. Context window size selection can affect performance.	Addresses the sparsity and lack of semantic meaning in traditional methods like BoW by learning dense word embeddings. ~ Addresses sparsity and semantic issues	
		Word	Example Embedding	Description			
		company	[0.25, -0.15, 0.35, 0.4, -0.25, 0.2, ...]	Word vector built from character n-grams, capturing subword information.			
		list	[0.1, -0.3, 0.5, -0.2, 0.25, 0.3, ...]	Effective for morphologically rich languages and handling rare words/misspellings.			
		numbers	[-0.2, 0.4, 0.3, -0.1, 0.2, 0.1, ...]	Produces high-quality embeddings by considering subword units.			
Global Vectors for Word Representation (GloVe)	2014	Global occurrence	GloVe is a word embedding model that combines global word co-occurrence statistics with local context-based learning. It constructs word vectors based on the probabilities of words co-occurring in a given context. ~ Each word is represented by a dense vector that captures both global word co-occurrence statistics and local context. These embeddings	Captures both global and local context. Produces interpretable and high-quality embeddings. Scales well with large corpora.	Requires significant memory for large vocabularies. Pre-training can be time-consuming.	Addresses the limitations of both BoW and Word2Vec by leveraging global co-occurrence statistics and local context, providing a more comprehensive understanding of word relationships. ~ Combines global statistics with	Similar words: "fantastic", "excellent" Captures global and local context, predicts similar words based on overall usage statistics.

			are often more interpretable.			local context	
		Word	Example Embedding	Description			
		company	[0.3, -0.2, 0.4, 0.7, -0.1, 0.5, ...]	Combines global co-occurrence statistics with local context learning.			
		list	[-0.1, 0.3, -0.4, 0.2, 0.6, -0.3, ...]	Produces interpretable and high-quality embeddings capturing both global and local information.			
		numbers	[0.2, 0.4, -0.5, 0.1, 0.3, -0.2, ...]	Scales well with large corpora.			
FastText	2016	n-gram (go read about it, its super cool)	FastText, developed by Facebook's AI Research (FAIR) lab, extends the Word2Vec model by representing words as n-grams of characters, thus capturing subword information and morphological structure. ~ Each word is represented by a dense vector built from character n-grams , capturing subword information and handling morphologically rich languages and rare words effectively.	Effective for morphologically rich languages. Handles rare words and misspellings well. Produces high-quality embeddings.	Slightly slower to train than Word2Vec models. Requires more memory due to n-gram representation.	Addresses the limitation of Word2Vec models in handling rare words and misspellings by considering subword information. Addresses sparsity and semantic issues ~Addresses rare words and subword info	Similar words: "gr8", "excellent" Captures subword information, handles morphological variations, predicts similar words effectively.
		Word	Example Embedding	Description			
		company	[0.25, -0.15, 0.35, 0.4, -0.25, 0.2, ...]	Word vector built from character n-grams, capturing subword information.			
		list	[0.1, -0.3, 0.5, -0.2, 0.25, 0.3, ...]	Effective for morphologically rich languages and handling rare words/misspellings.			
		numbers	[-0.2, 0.4, 0.3, -0.1, 0.2, 0.1, ...]	Produces high-quality embeddings by considering subword units.			

Seq2seq – Encoder-Decoder Neural Network

Original paper:

<https://arxiv.org/abs/1409.3215>

<https://arxiv.org/pdf/1409.3215>

Youtube video by Statquest:

<https://www.youtube.com/watch?v=L8HKweZIOmg&list=PLbIh5JKOoLUixGDQs4LFFD--41Vzf-ME1&index=18>

- The Youtube video shows Encoder-Decoder to translate English to Spanish.

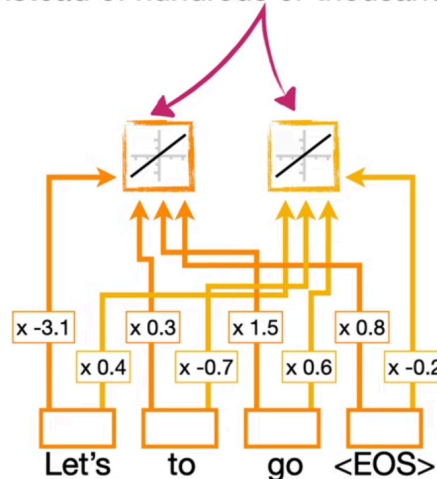
Tutorial by Bentrevett : <https://github.com/bentrevett/pytorch-seq2seq/tree/rewrite>

Initial note:

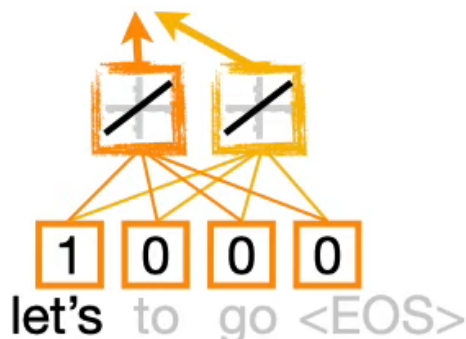
1. Words are now translated into tokens. Tokens does not mean words but a representative of words. Token can be anything determined by the word embedding method. Hence words are referred as “elements” and called “tokens”



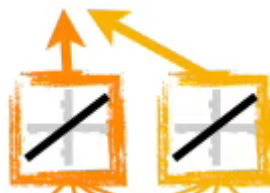
ALSO NOTE: In this example, we're just creating **2** embedding values per **Token** instead of hundreds or thousands.



2. 4:48
3. Orange with 2 / – are the **embedding layer** – serve as input for first layer. As shown,



4. Embedding value refer to this, 2 Embedding values refer to,



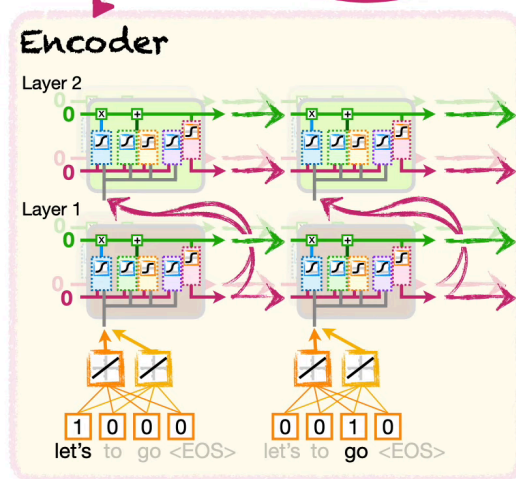
5. Hence, this embedding layer has 2 embedding values for 4 tokens. Embedding layer is the first layer for the Encoder and Decoder.
6. Embedding Layer for Encoder and Decoder have different tokens, parameters, hence different embedding value for each token.

7. <EOS> stands for End Of Sentence. But sometimes people use <SOS> or Start Of Sentence.

Encoder

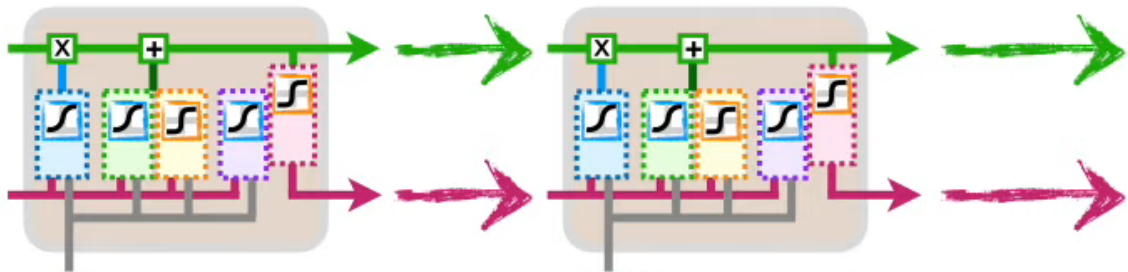


And now we're done creating the **Encoder** part of the **Encoder-Decoder** model.



7:38

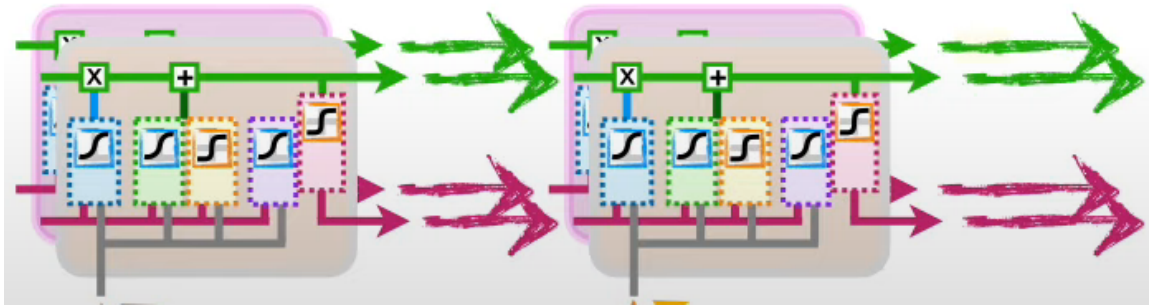
- One LSTM cell refer to this,



6:05

This is a single LSTM cell being unfolded once.

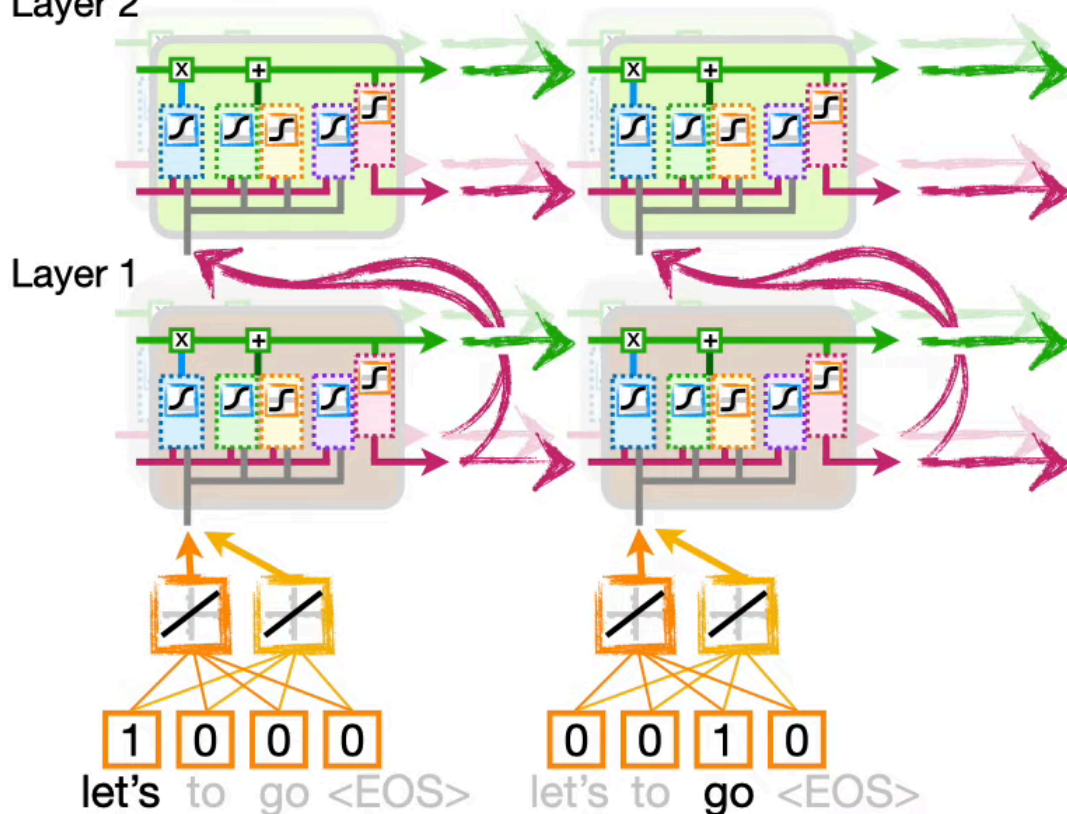
- Example shown above has 2 Layers of 2 LSTM cells;
One layer has 2 LSTM cells, 1 being hidden. (pink)



6:30

- One LSTM cell shares the same weights and biases for all unrolled (visually sequentially connected)
- Different LSTM cells, even in same layer, has different weights and biases. Example pink and orange have different weights and biases. Hence the colour code.
- In order to have take more embedded values (read more words), more (hidden state) LSTM cells are added in Layer 1 – which directly connects to the input from embedding layer.
- In order to add more weights and biases (increase complexity of model) to fit the model, additional layers of LSTM are added.

Layer 2



6:56

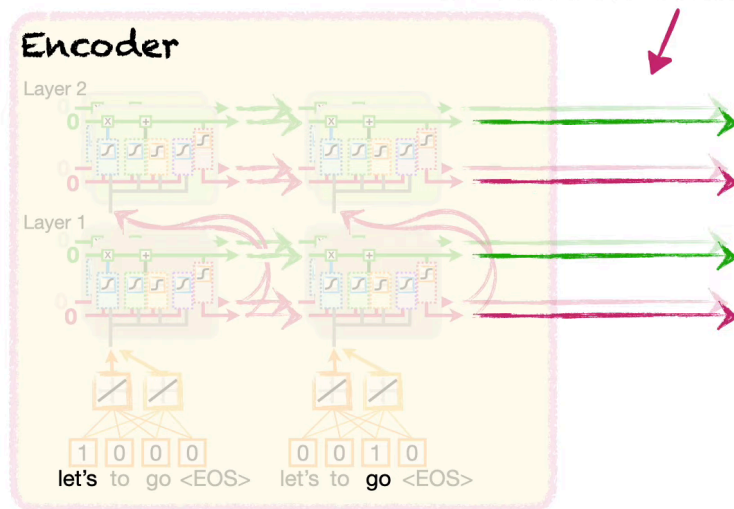
- Output values from **short-term memory** are fed as input for the next layer. This is possible because the number of hidden states (LSTM cells or nodes) are the same for all layers.

However, this also depends on architecture as some may add or reduce the number of nodes down the layers and have different ways to handle to increase or reduction.

- Encoder encodes the input sentence (e.g. Let's go), into a collection of long and short term memories (called cell and hidden states) 7:51
- Terminology:



The last long and short term memories (the cell and hidden states) from both layers of the **LSTM** cells in the **Encoder** are called the **Context Vector**.



8:07

*The Encoder encodes the input sentence, "Let's go", into the **Context Vector**.*

Essentially an Encoder is a layer with an Embedding Layer and a long list of LSTM cells on different layers.

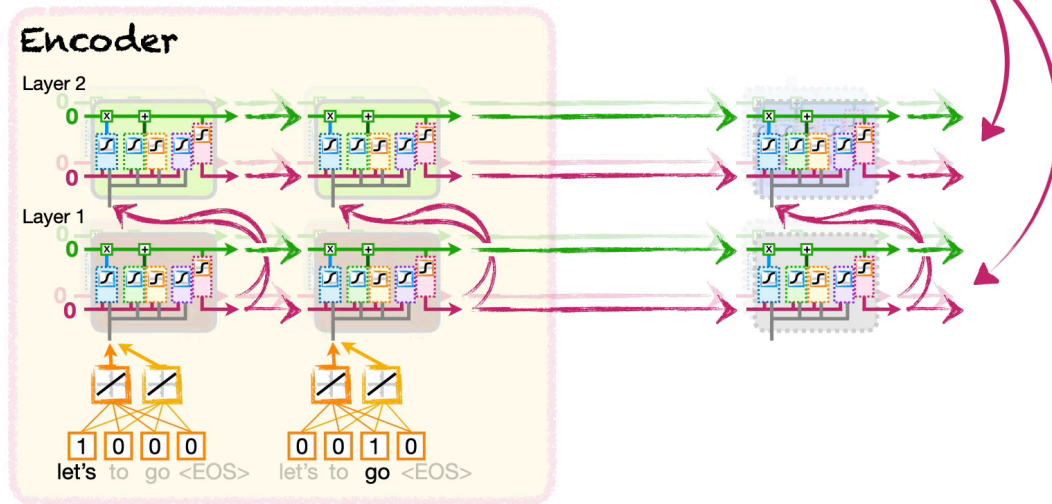
Decoder

- Connect the Context Vector to the Decoder's LSTM. Context Vector is used to initialize short and long-term memories (both cell and hidden state) in Decoder's LSTM.



To exit full screen, press Esc

...that, just like in the **Encoder**, have **2** layers, and each layer has **2** cells.



8:43

- LSTMs from Decoder have different parameters than the Encoder's.
- The first layer of Decoder is also an Embedding Layer. In this example, this Embedding Layer creates embedding values for the Spanish word.



9:18

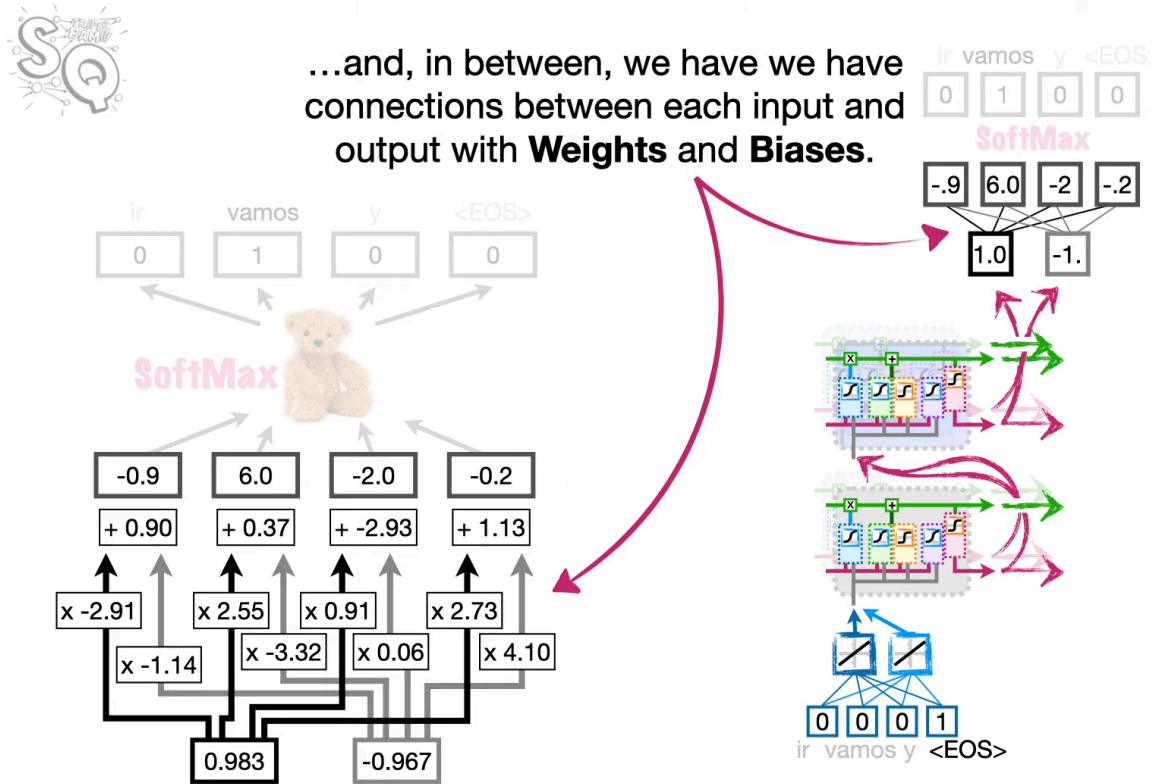
- The Decoder's Embedding Layer starts with <EOS> token. This is predicted token, not necessarily the correct translation to the English counterpart.
- NOTE:

The top layer (near output)'s SHORT-TERM MEMORY are sent to a Fully Connected Layer (the simple categorizer NN model) with its own parameters.

The FCN layer has 2 inputs for the 2 values that come from the top layer output. Go through some parameters and then it has 4 outputs, one for each token in the Spanish vocabulary.

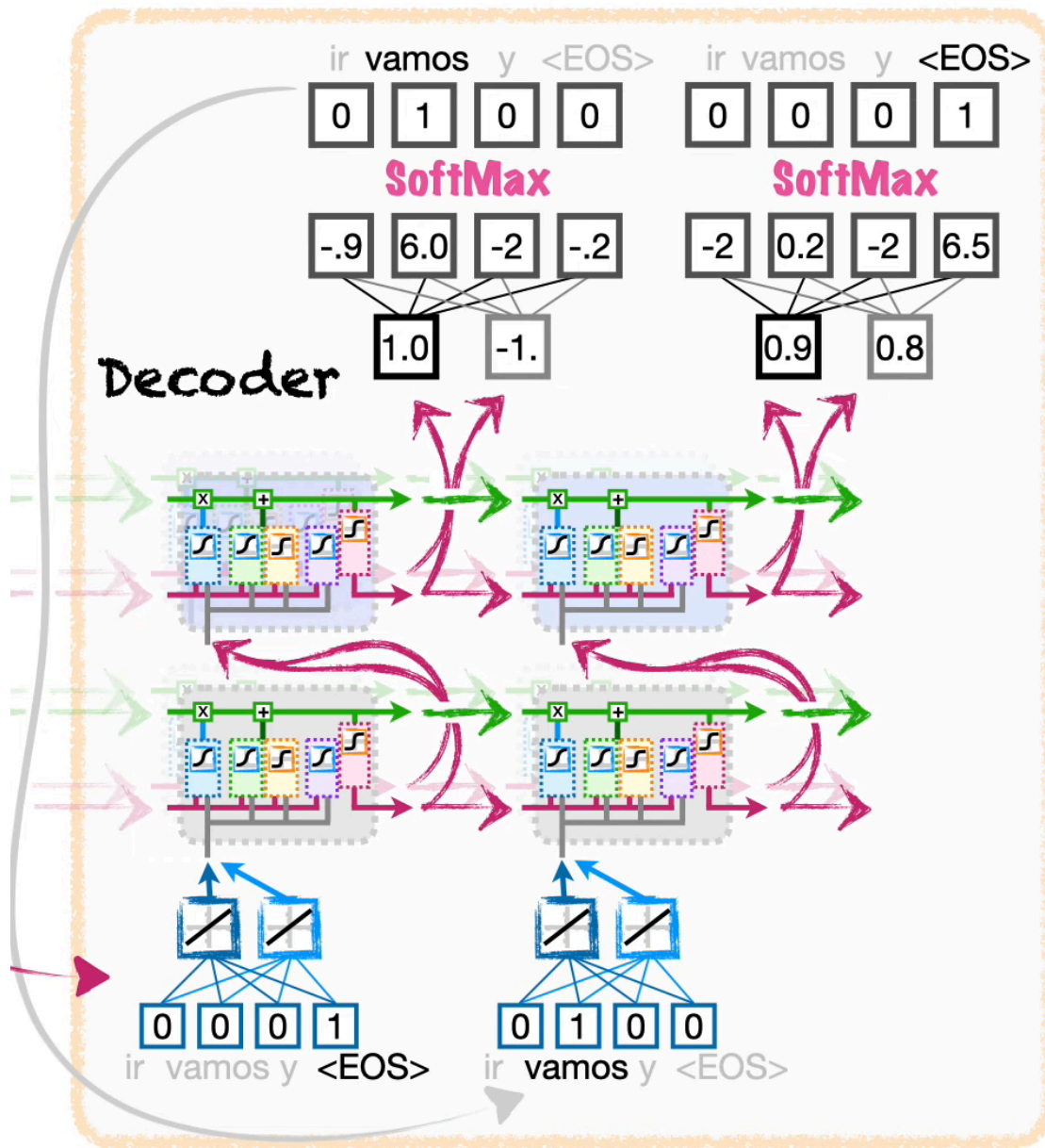
Then sent to a SoftMax activation function the output word.

Visually,



11:06

- The Decoder does not stop until it decodes (outputs) a $\langle EOS \rangle$ or hit some maximum output length.
- So the LSTM cells are unrolled to input the second word,



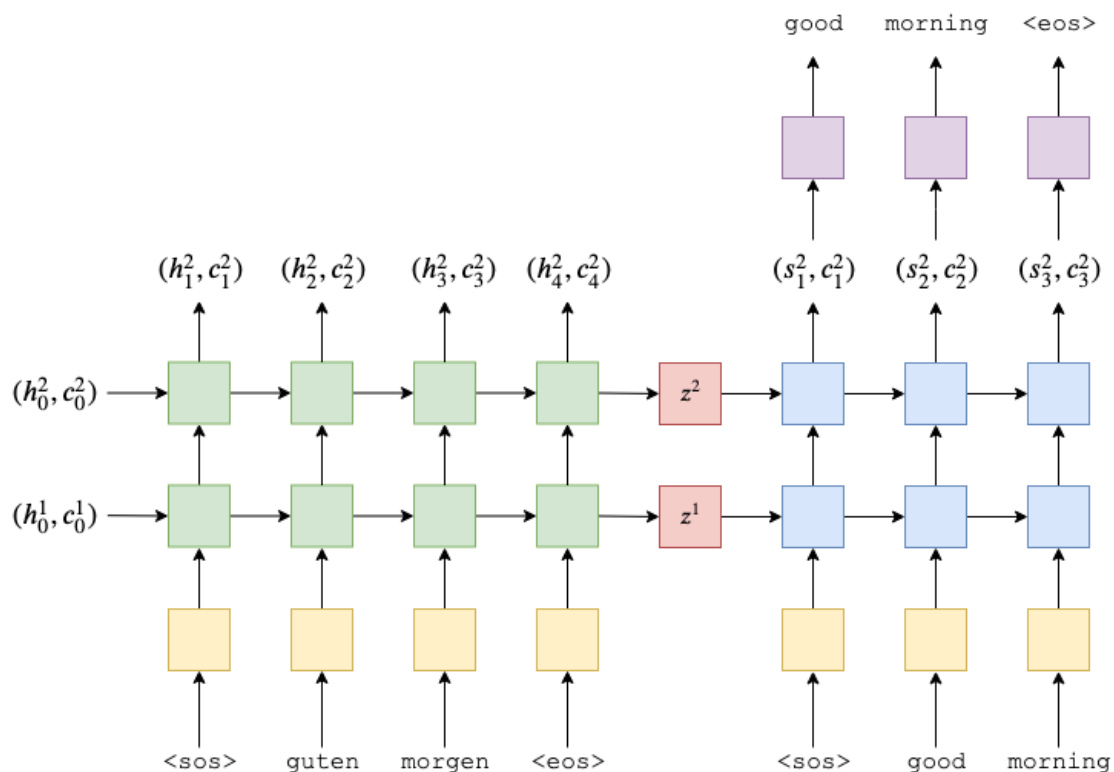
During training

- When **training** the **Encoder-Decoder**, instead of using the predicted token (blue) as input to the Decoder LSTMs, we **use the known, correct token** as input for the unrolled LSTMs. Check out 13:00.
- Instead of just predicting tokens until Decoder predicts <EOS> token, each output phrase ends where the known phrase ends
 ie. If it is supposed to predict <EOS> but Decoder decoded to be something else, training will stop.

- Plugging in known words and stopping at known phrase length, rather than using predicted tokens for everything, is called **Teacher Forcing**. 14:25.
- For this implementation, we have to ensure that the number of layers and hidden (and cells) dimensions are equal in the `Encoder` and `Decoder`.
- This is not necessarily required (to have same number of layer or same hidden dimension sizes in a sequence-to-sequence model).
- However, if different number of layers, then need to figure out how to handle them.
- e.g. if encoder has 2 layers but decoder has 1 layer.
- Do we average the two context vectors output by the decoder?
- Do we pass both through a linear layer?
- Do we only use context vector from highest level? etc.
- `forward` method takes the source sentence, target sentence and a teacher-forcing ratio.
- teacher-forcing ratio is used when training our model. It is an algorithm for training weights of RNN involving feeding observed sequence values (i.e. ground-truth samples) back into RNN after each step, thus forcing RNN to stay close to the ground-truth sequence.

The Seq2Seq model looks like this. Credit to

<https://github.com/bentrevett/pytorch-seq2seq/blob/rewrite/1%20-%20Sequence%20to%20Sequence%20Learning%20with%20Neural%20Networks.ipynb>



- receives the input/source sentence
- use the encoder to produce the context vectors
- use the decoder to produce the predicted output/target sentence

Extra Note 1:

Why use multiple embedding values for a single word?

1. Polysemy

- **Polysemy** refers to the phenomenon where a single word has multiple meanings. For example, the word "bank" can mean a financial institution or the side of a river. Using multiple embeddings allows the model to represent these different meanings.

2. Contextual Information

- Modern NLP models, like BERT and GPT, use contextual embeddings, where the embedding of a word depends on its surrounding words. This way, the same word can have different embeddings depending on its context, allowing the model to better understand the nuances in meaning.

3. Improved Representation

- Multiple embeddings can provide a richer and more detailed representation of words, capturing subtle differences and relationships that a single embedding might miss. This can lead to improved performance on various NLP tasks.

4. Dynamic Contextualized Embeddings

- Models like ELMo (Embeddings from Language Models) and BERT generate dynamic, context-dependent embeddings for each word. This means that a word will have different embeddings in different sentences, reflecting its specific use in each context.

Examples of Models Using Multiple Embeddings

1. **ELMo**: Generates word embeddings by considering the entire sentence, producing different embeddings for the same word depending on its context.
2. **BERT**: Uses a transformer architecture to produce embeddings where each word's representation is influenced by every other word in the sentence, resulting in context-specific embeddings.
3. **GPT**: Similar to BERT, it generates embeddings that are context-dependent, capturing the nuanced meanings of words in different scenarios.